

GDPRuler: A Trusted GDPR Monitor for Cloud Data Systems

Dimitrios Stavrakakis
dimitrios.stavrakakis@tum.de
TU Munich
Munich, Germany

Masanori Misono
masanori.misono@in.tum.de
TU Munich
Munich, Germany

Julian Pritzi
julian.pritzi@tum.de
TU Munich
Munich, Germany

Harshavardhan Unnibhavi
harshavardhan.unnibhavi@tum.de
TU Munich
Munich, Germany

Nuno Santos
nuno.m.santos@tecnico.ulisboa.pt
INESC-ID / Instituto Superior Técnico
Lisbon, Portugal

Pramod Bhatotia
pramod.bhatotia@tum.de
TU Munich
Munich, Germany

A ARTIFACT APPENDIX

A.1 Abstract

This artifact contains source code, workloads, scripts, and plotting utilities for reproducing the evaluation of GDPRULER: A TRUSTED GDPR MONITOR FOR CLOUD DATA SYSTEMS. It supports end-to-end execution and reproduction of (i) YCSB performance for bare-metal and CVM setups, (ii) GDPR query workloads including metadata-index experiments, (iii) CVM-based microbenchmarks, and (iv) logging microbenchmarks for storage/performance impact analyses. The artifact is delivered as a Git repository with submodules and a single AE driver script that automates setup, execution, and plotting. Outputs include CSV-formatted measurements and paper figures.

A.2 Description & Requirements

The repository root contains the controller code, workload generators, benchmark drivers, CVM setup scripts, and plotting scripts. Important directories/files:

- README.md: repository structure and build instructions.
- AE/README.md: Artifact Evaluation (AE) roadmap and one-command reproduction entry point.
- AE/run_ae.sh: end-to-end AE orchestration script.
- CVM_setup/: scripts and images for CVM setup.
- evaluation/bare_metal: bare-metal experiment scripts.
- evaluation/VM: CVM experiment scripts.
- evaluation/microbenchmarks/CVM_eval: submodule with CVM-based microbenchmarks.
- evaluation/plots/: scripts for generating paper plots.
- gdpr-logger/: logging subsystem and its microbenchmarks.
- ycsb_trace_generator/: workload generation subsystem.
- workload_traces/: generated and test workload traces.

A.2.1 Security, privacy, and ethical concerns. The artifact executes benchmarking workloads, builds native binaries, and launches CVMs; therefore, ideally, it should be run on an isolated test machine. The AE runner performs a destructive submodule reset (removing and reinitializing existing submodule directories). CVM-related scripts may require elevated privileges depending on local virtualization/network setup. Evaluators should not run this workflow on production systems or systems containing sensitive data.

A.2.2 How to access. All the project code, along with instructions on how to evaluate and build the software, is available in:

- Archival artifact: [Zenodo link](#)
- Development mirror: [Github link](#)

A.2.3 Hardware dependencies. The testbed used to conduct our experiments is equipped with an AMD EPYC 7713P CPU (64 cores, hyperthreading disabled) and 1024 GB of DDR4 DRAM (16×64 GB DIMMs).

Required hardware for artifact execution:

- x86_64 Linux server.
- Multi-core CPU (> 32 cores), sufficient RAM (> 128 GB), and storage space to support multi-hour benchmark runs.
- AMD-SEV-SNP-capable platform for CVM experiments.
- Fast local storage recommended for benchmark KVS data and logs.

Note: Partial reproduction (bare-metal-only plotting and non-CVM components) is possible without SEV-SNP.

A.2.4 Software dependencies. On our testbed servers, the host runs NixOS 25.05 with an AMD SEV-SNP-enabled Linux kernel (v6.16.11), while the CVMs run Ubuntu 22.04 with a Linux kernel version v6.5.0. The nix environment exposes the required build and plotting dependencies at the versions pinned by the repository.

Required software for artifact execution:

- Linux shell with bash, git, cmake, make, python3.
- nix package manager (the artifact uses flake.nix and a committed flake.lock; nix develop provides the pinned toolchain).
- VM/CVM tools used by the project setup (e.g., expect, virt-customize) for CVM flow.

A.2.5 Benchmarks. For AE, we use the following workloads:

- YCSB-derived workloads (A/B/C/D/F).
- GDPR query workloads inspired from [GDPRBench](#) [2].
- iperf and fio workloads from [CVM_eval](#) [1].
- Logger microbenchmarks in gdpr-logger submodule.

A.3 Set Up

A.3.1 Installation. The setup below covers the initial build steps used by the artifact; the same setup is also performed by the bundled AE/run_ae.sh wrapper that orchestrates a complete benchmark execution.

First, clone the repository or retrieve and unzip the archived version and initialize the submodules:

```
$ git clone git@github.com:dimstav23/GDPRuler.git
$ git submodule update --init --recursive # init the submodules
```

From repository root, run:

```
# Get into the dev environment
$ nix develop
```

```
# Generate the YCSB workloads (5-10 mins)
$ cd ycsb_trace_generator
$ bash workload_generator.sh
# Build the logging subsystem
$ cd ../gdpr-logger
$ mkdir -p build && cd build
$ cmake .. -D CMAKE_BUILD_TYPE=Release -D BUILD_BENCHMARKS=ON
$ make -j$(nproc)
# Test the building of the GDPR controller
$ cd ../../controller
$ cmake -S . -B build -D CMAKE_BUILD_TYPE=Release
$ cmake --build build
# Compile Redis
$ cd ../KVs/redis
$ make BUILD_TLS=yes MALLOC=libc
# Build the required CVM image
$ cd ../../CVM_setup/AMDSEV
$ bash ./build.sh ovmf
$ cd ../
$ bash GDPRuler_VMs_setup.sh
```

Note: By default, AE/run_ae.sh performs this setup and then continues with the evaluation workflow.

Networking and storage configuration notes. The default setup assumes bridge/network and storage paths that may need adaptation on a new machine:

- Bridge/IP defaults: virbr0 and 192.168.122.0/24, with guest IP 192.168.122.48. The relevant files that define such configurations are:
 - evaluation/common.sh
 - evaluation/bare_metal/config.sh
 - evaluation/VM/config.sh
 - CVM_setup/GDPRuler_VMs_setup.sh
 - CVM_setup/network_configs/*.yaml
- The helper CVM_setup/prepare_net_cfg.sh can rewrite netplan prefix based on bridge subnet, but host-side CVM IP constants must still match the final guest IP.
- Storage defaults are configured in evaluation/common.sh: NVME_DEVICE, MOUNT_POINT, DB_DUMP_DIR, CTL_DUMP_DIR.
- prepare_storage_host() of evaluation/common.sh mounts a device and creates a filesystem (mkfs) on the device pointed by NVME_DEVICE; evaluators should set this variable carefully for their hardware.
- The -scratch-base option in AE/run_ae.sh controls where plotting scripts read results; it does not override all storage constants inside evaluation scripts.

A.3.2 Basic test. For a minimal bare-metal functionality test, after the setup phase, run the following from the repository root:

```
$ cd KVs/redis/src
$ ./redis-server --protected-mode no --unixsocketperm 700 --
  unixsocket /tmp/redis.sock &
$ cd ../../../../
$ python3 scripts/GDPRuler.py --db redis --logpath <file_directory>
  &
$ python3 scripts/client.py --workload workloada_small --clients 1 --
  config ./configs
```

Expected outcome: the Redis server starts, the GDPR controller starts and connects successfully to the server via a unix socket, and the client connects to the controller and completes one small workload run without errors. Different types/sizes of workloads can be passed as command line parameters.

A.4 Evaluation Workflow

A.4.1 Major claims. Our results are organized into three claim groups that align with the main figures and tables in the paper.

- (C1): GDPRULER performance under GDPR compliance. This includes the KVS workload analysis and the GDPR query analysis, corresponding mainly to Figures 5–7.
- (C2): GDPRULER’s storage efficiency. This includes the logging layer impact, the logging layer performance, the KVS meta-data storage overhead, and the audit-log storage requirements, corresponding mainly to Figures 8–9 and Table 7.
- (C3): Data I/O subsystem performance. This includes CVM storage I/O performance and network I/O performance, corresponding mainly to Figures 10–11 of the paper.

A.4.2 Experiments. All experiments below can be executed manually with the explicit commands shown per experiment; the same sequence is also packaged end-to-end by AE/run_ae.sh.

- (E1): [Performance under GDPR compliance] [approximately 2-3 days of runtime for a complete configuration run using paper’s workload sizes]

Preparation: Ensure the host is configured as described in the previous subsection (Nix, bridge/network, storage devices/paths, and CVM requirements). Make sure that you are in a nix shell with all the software dependencies present.

Execution: Run the following commands from the repository root:

```
# Core bare-metal and CVM experiments for Redis/RocksDB
# across configurations
# Note: this step also includes logging workloads for C2
$ cd evaluation
$ bash evaluation_runner.sh
# Generate the plots for C1
$ cd plots
$ python3 generate_ycsb_performance_plot.py \
  --output_dir paper_plots \
  --vm_results /scratch/<user>/GDPRuler/evaluation/VM/
  results/ \
  --bare_metal_results /scratch/<user>/GDPRuler/evaluation/
  bare_metal/results/
$ python3 generate_gdpr_queries_plots.py \
  --output_dir paper_plots \
  --vm_results /scratch/<user>/GDPRuler/evaluation/VM/
  results/ \
  --bare_metal_results /scratch/<user>/GDPRuler/evaluation/
  bare_metal/results/
```

Results: Verify that the CSV-formatted results are placed in evaluation/bare_metal/results (for the bare-metal) and evaluation/VM/results (for the CVM); verify that the generated YCSB/GDPR-query plots are placed in evaluation/plots/paper_plots. These outputs support the KVS workload and GDPR query claims (Figures 5–7).

- (E2): [Storage efficiency] [approximately 10 hours runtime]

Preparation: Same as E1; logging experiments rely on the storage mount and the monitor-enabled workloads generated by the evaluation scripts in E1.

Execution: Run the following commands from the repository root to execute the logging microbenchmarks:

```
$ cd gdpr-logger
$ mkdir -p build && cd build
```

```

$ cmake .. -D CMAKE_BUILD_TYPE=Release \
-D BUILD_EXAMPLES=ON -D BUILD_TESTING=ON -D
  BUILD_BENCHMARKS=ON
$ make -j$(nproc)
$ ./gdpruler_log_performance_benchmark
$ ./gdpruler_compression_rate_benchmark
# Generate the plots for C2
$ cd ../plot_scripts
$ python3 gdpruler_benchmark_plot.py \
--input_file ../build/gdpr_logger_benchmark_results.csv
# Generate the plots and print statistics to stdout for C2
$ cd ../../evaluation/plots
$ python3 generate_logging_plot_and_stats.py \
--output_dir paper_plots \
--vm_results /scratch/<user>/GDPRuler/evaluation/VM/
  results \
--bare_metal_results /scratch/<user>/GDPRuler/evaluation/
  bare_metal/results

```

Results: The `gdpr_logger_benchmark_results.csv` in the build directory of the logger contains the results, and the plot outputs are in `evaluation/plots/paper_plots`. The plotting script also prints the storage tables/statistics used for Figures 8–9 and Table 7.

(E3): [Data I/O subsystem performance] [approximately 5-6 hours runtime]

Preparation: Same as E1; CVM setup requires the bridge, guest IP, and storage paths described above.

Execution: Run the following commands from the repository root:

```

$ cd ../evaluation/microbenchmarks
$ bash run_CVM_eval_benchmarks.sh

```

Results: Inspect the CVM-eval outputs and plots generated by `run_CVM_eval_benchmarks.sh`. They are placed in the plot directory in the `CVM_eval` submodule; these support the data I/O subsystem claims in Figures 10–11.

A.5 Notes on Reusability

The artifact can be reused to evaluate additional controller configurations by adjusting evaluation config scripts (e.g., workload size, client counts, encryption/logging settings) and reusing the same run/plot pipeline. For faster sanity checks, evaluators can switch workload type to `small` in evaluation config files and restore default values for full runs.

Practical note. In some environments, CVM runs can fail if CVM-side network access is restricted (e.g., dependency fetches from GitHub during setup). This does not affect the structure of the artifact, but it may require local infrastructure/network adjustments.

A.6 Version

Based on the LaTeX template for Artifact Evaluation V20220926.

REFERENCES

- [1] Masanori Misono, Dimitrios Stavrakakis, Nuno Santos, and Pramod Bhatotia. Confidential VMs Explained: An Empirical Analysis of AMD SEV-SNP and Intel TDX. *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, 8(3), 2024.
- [2] Supreeth Shastri, Vinay Banakar, Melissa Wasserman, Arun Kumar, and Vijay Chidambaram. Understanding and benchmarking the impact of gdpr on database systems. *Proc. VLDB Endow.*, 13(7):1064–1077, March 2020.